

.NET re

ASP.NET re



.NET Cre

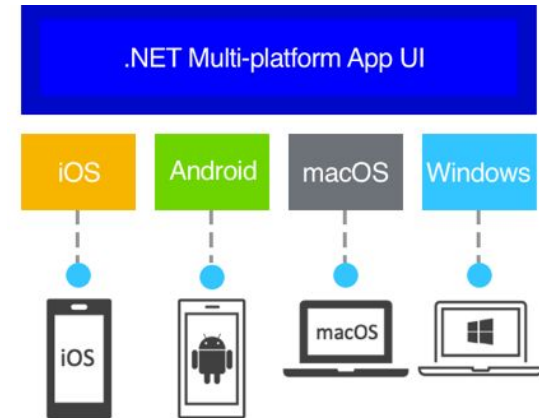


.NET – A unified platform



.NET Core Overview

- ❖ .NET Core is a new version of .NET Framework, which is a free, open-source, general-purpose development platform maintained by Microsoft. It is a cross-platform framework that runs on Windows, macOS, and Linux operating systems.

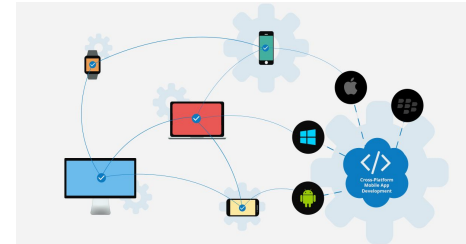
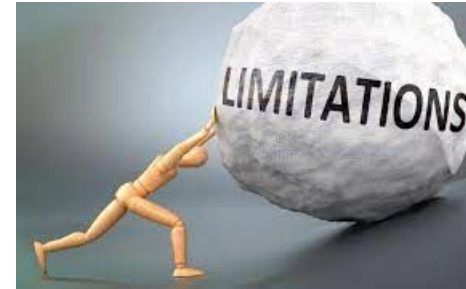


- ❖ .NET Core Framework can be used to build different types of applications such as mobile, desktop, web, cloud, IoT, machine learning, microservices, game, etc.

Why .NET Core?

Limitations with the .NET Framework:

- ❖ it only runs on the Windows platform.
- ❖ Use different .NET APIs for different Windows devices such as Windows Desktop, Windows Store, Windows Phone, and Web applications.
- ❖ the .NET Framework is a machine-wide framework. Any changes made to it affect all applications taking a dependency on it.



Limitlessness with the .NET Core:

- ❖ Today, it's common to have an application that runs across devices; a backend on the web server, admin front-end on windows desktop, web, and mobile apps for consumers. So, there is a need for a single framework that works everywhere.
- ❖ The main objective of .NET Core is to make .NET Framework open-source, cross-platform compatible that can be used in a wide variety of verticals, from the data center to touch-based devices.

.NET Core Characteristics

- **Open-source Framework:** .NET Core is an [open-source framework](#) maintained by Microsoft and available on GitHub under [MIT](#) and [Apache 2](#) licenses. It is a [.NET Foundation project](#).

You can view, download, or contribute to the source code using the following GitHub repositories:

- Language compiler platform Roslyn: <https://github.com/dotnet/roslyn>
 - .NET Core runtime: <https://github.com/dotnet/runtime>
 - .NET Core SDK repository. <https://github.com/dotnet/sdk>
 - ASP.NET Core repository. <https://github.com/dotnet/aspnetcore>
-
- **Cross-platform** :.NET Core runs on Windows, macOS, and Linux operating systems.
 - **Consistent across Architectures:** Execute the code with the same behavior in different instruction set architectures, including x64, x86, and ARM.
 - **Wide-range of Applications:** Various types of applications can be developed and run on .NET Core platform such as mobile, desktop, web, cloud, IoT, machine learning, microservices, game, etc.
 - **Supports Multiple Languages:** You can use C#, F#, and Visual Basic programming languages to develop .NET Core applications. You can use your favorite IDE, including Visual Studio 2017/2019, Visual Studio Code, Sublime Text, Vim, etc.
 - **Modular Architecture:** .NET Core supports modular architecture approach using NuGet packages. There are different NuGet packages for various features that can be added to the .NET Core project as needed.
 - **CLI Tools:** .NET Core includes CLI tools (Command-line interface) for development and continuous-integration.





ASP.NET Core



ASP.NET Core Overview

- ❖ ASP.NET Core is the new version of the ASP.NET web framework mainly targeted to run on .NET Core platform.
- ❖ ASP.NET Core is a free, open-source, and cross-platform framework for building cloud-based applications, such as web apps, IoT apps, and mobile backends. It is designed to run on the cloud as well as on-premises.
- ❖ Same as .NET Core, it was architected modular with minimum overhead, and then other more advanced features can be added as NuGet packages as per application requirement. This results in high performance, require less memory, less deployment size, and easy to maintain.
- ❖ ASP.NET Core is an open source framework supported by Microsoft and the community, so you can also contribute or download the source code from the [ASP.NET Core Repository on Github](#).
- ❖ ASP.NET 3.x runs only on .NET Core 3.x, whereas ASP.NET Core 2.x runs on .NET Core 2.x as well as .NET Framework.



Why ASP.NET Core?

- **Supports Multiple Platforms:** ASP.NET Core applications can run on Windows, Linux, and Mac. So you don't need to build different apps for different platforms using different frameworks.
- **Fast:** ASP.NET Core no longer depends on System.Web.dll for browser-server communication. ASP.NET Core allows us to include packages that we need for our application. This reduces the request pipeline and improves performance and scalability.
- **IoC Container:** It includes the built-in IoC container for automatic dependency injection which makes it maintainable and testable.
- **Integration with Modern UI Frameworks:** It allows you to use and manage modern UI frameworks such as AngularJS, ReactJS, Umber, Bootstrap, etc. using Bower (a package manager for the web).
- **Hosting:** ASP.NET Core web application can be hosted on multiple platforms with any web server such as IIS, Apache etc. It is not dependent only on IIS as a standard .NET Framework.
- **Code Sharing:** It allows you to build a class library that can be used with other .NET frameworks such as .NET Framework 4.x or Mono. Thus a single code base can be shared across frameworks.
- **Side-by-Side App Versioning:** ASP.NET Core runs on .NET Core, which supports the simultaneous running of multiple versions of applications.
- **Smaller Deployment Footprint:** ASP.NET Core application runs on .NET Core, which is smaller than the full .NET Framework. So, the application which uses only a part of .NET CoreFX will have a smaller deployment size. This reduces the deployment footprint.



Components of MVC

MVC stands for Model, View and Controller.

The Model – A set of classes that describes the data you are working with as well as the business logic.

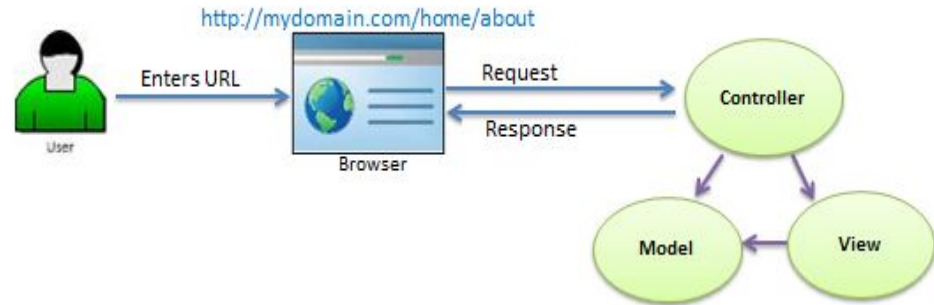
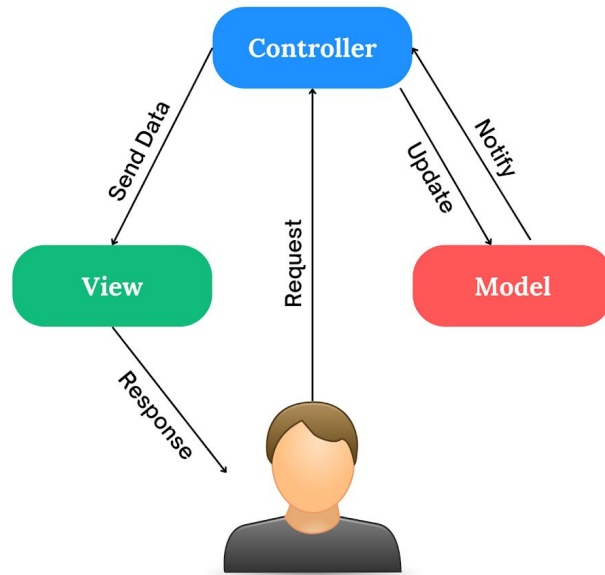
The View – Defines how the application's UI will be displayed. It is a pure HTML, which decides how the UI is going to look like.

The Controller – A set of classes that handles communication from the user, overall application flow, and application-specific logic.

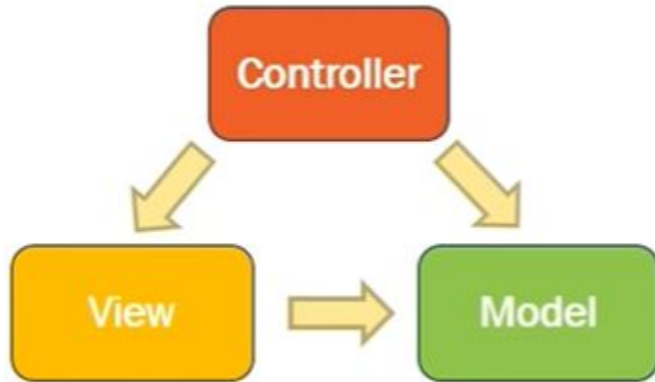


Architecture of MVC

MVC Architecture



ASP.NET Core MVC vs Razor Pages



.NET Core Command-Line Interface

Basic Commands	Description
new	Creates a new project, configuration file, or solution based on the specified template.
restore	Restores the dependencies and tools of a project.
build	Builds a project and all of its dependencies.
Run	Runs source code without any explicit compile or launch commands.
publish	Packs the application and its dependencies into a folder for deployment to a hosting system.
test	Executes unit tests.
vtest	Runs tests from the specified files.
pack	Packs the code into a NuGet package.
clean	Cleans the output of a project.
sln	Modifies a .NET Core solution file.
help	Display help on the specified command
store	Stores the specified assemblies in the runtime package store.



.NET Core Command-Line Interface

Let's create, restore, build, and run .NET Core console application using command-line interface without using VS Code.

- The following creates new mvc project in the current directory with the same name as current directory.

dotnet new mvc

- The following command creates a new mvc project named MyApp. The -n or --name option species the name of a project.

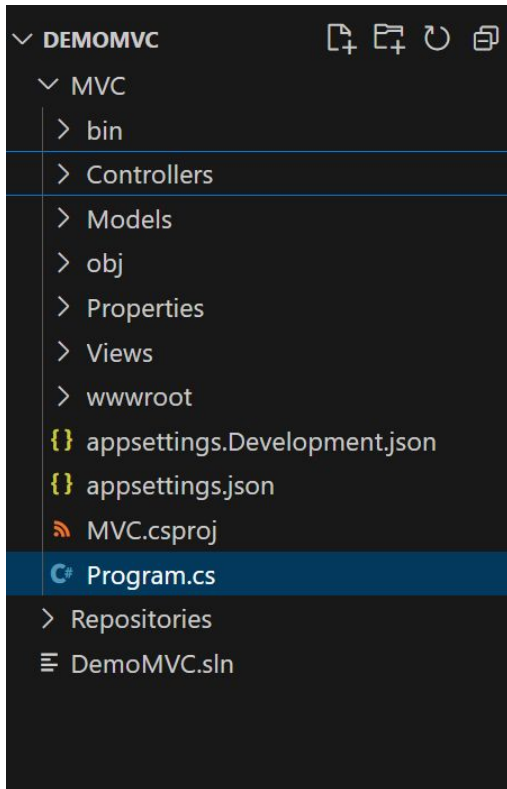
dotnet new mvc -n MyConsoleApp

- The following command creates a new mvc project named MyApp to MyProjects directory. The -o or --output option is used to specify an output directory where the project should be generated.

dotnet new mvc -n MyConsoleApp -o C:\MyProject



Project Structure



- **bin/** → Stores compiled assemblies (.dll files) and dependencies after a build.
- **Controllers/** (Handles requests, contains logic)
- **Models/** (Data structures, business logic)
- **obj/** → Contains intermediate files, temporary build data, and metadata used during the compilation process.
- **Views/** (UI templates with Razor) **Views/Shared/** (Common UI components)
- **wwwroot/** (Static files like CSS, JS)
- **Middleware & Config Files** (Startup.cs, appsettings.json)
- **Repositories/** follow the **Repository Pattern**, acting as an abstraction layer between the data access logic and business logic.

Output

EmptyMVC [Home](#) [Privacy](#)

Welcome

Learn about [building Web apps with ASP.NET Core](#).

© 2024 - EmptyMVC - [Privacy](#)



appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=MyDB;User
    Id=myuser;Password=mypassword;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "MyCustomSettings": {
    "FeatureEnabled": true,
    "MaxUsers": 100
  }
}
```

ConnectionStrings → Stores database connection strings.

Logging → Configures logging levels for different namespaces.

AllowedHosts → Defines which hosts are allowed (use * for any).

Custom Settings → You can define your own settings under custom keys.



Add New Controller

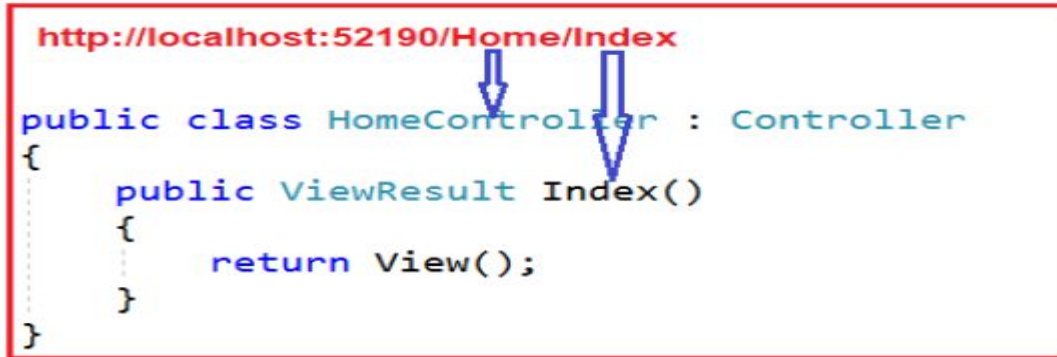
- The following creates new mvc controller in the current directory.
cd Controllers
dotnet new mvcontroller TestController
cd ..
- The following command creates a new directory in view for controller.
cd Views
mkdir Test
cd ..



MVC Routing

Routing module is responsible for mapping the incoming browser requests (i.e. the incoming URL) to a particular MVC controller action method.

This mapping is done by the routing rules defined for your application. For example, if we issue a request to the “/Home/Index” URL, then it is the Index action method of Home Controller class which is going to handle the request



Transporting data from the controller to view

A controller is responsible for handling overall communication between model and view. It will access data from the model and send to View. They are:

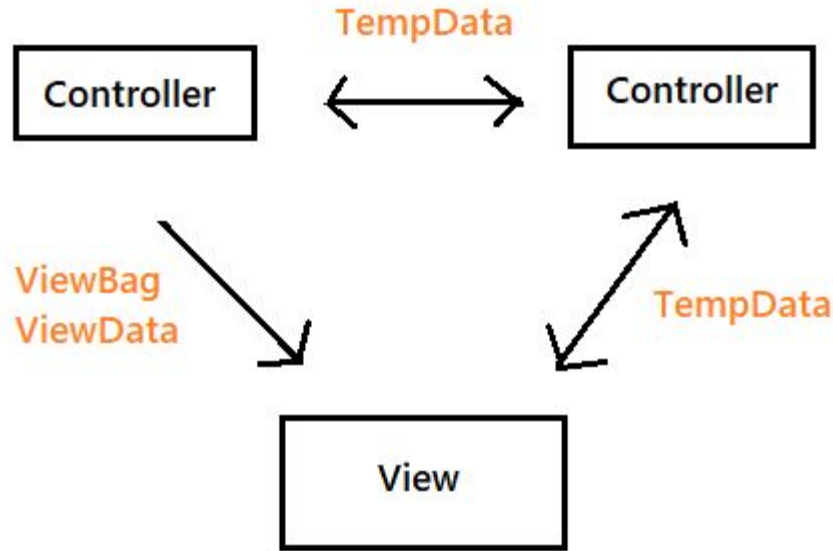
1. ViewData
2. ViewBag
3. TempData
4. Session

Pass data from the controller to view; Pass data from one controller to another controller;

Pass data from one action to another action; Pass data between subsequent HTTP requests



Transporting data from the controller to view



- ViewBag
 - dynamic object to pass data from controller to view
- ViewData
 - dictionary object to pass data from controller to view
- TempData
 - dictionary object to pass data from one controller/action to another controller/action

View Data

- **Description:** A dictionary for passing data from the controller to the view.
- **Use:** Suitable for sending data that is only needed for the current request.
- **Typcasting:** Required when retrieving data, as values are stored as **object**.

```
public class DemoController : Controller
{
    // GET: Demo
    public ActionResult Index()
    {
        ViewData["EmployeeName"] = "Hitesh Patel";
        ViewData["Technology"] = new List<string>() { "ASP.Net",
"SQL Server", "MVC", "Angular" };
        return View();
    }
}
```

```
//Index.cshtml
@{
    ViewData["Title"] = "Employee Details";
}

<h1>@ViewData["Title"]</h1>

<p>Employee Name: @ViewData["EmployeeName"]</p>

<h2>Technologies:</h2>
<ul>
    @foreach (var tech in (List<string>)ViewData["Technology"])
    {
        <li>@tech</li>
    }
</ul>
```

O/P
Employee Details
Employee Name: Hitesh Patel

Technologies:
- **ASP.Net**
- **SQL Server**
- **MVC**
- **Angular**



View Bag

- **Description:** A dynamic property for passing data from the controller to the view, built on top of ViewData.
- **Use:** Convenient for passing data with less syntax and type checking.
- **Typecasting:** Not required when using the dynamic property, but casting may be needed if the data type is unknown.

```
public class DemoController : Controller
{
    // GET: Demo
    public ActionResult Index()
    {
        ViewBag.EmployeeName= "Hitesh Patel";
        ViewBag.Technology = new List<string>() { "ASP.Net",
"SQL Server", "MVC", "Angular" };
        return View();
    }
}
```

```
@{
    ViewBag.Title = "Employee Details";
}

<h1>@ViewBag.Title</h1>

<p>Employee Name: @ViewBag.EmployeeName</p>

<h2>Technologies:</h2>
<ul>
    @foreach (var tech in ViewBag.Technology as List<string>)
    OR
    @foreach (var technology in ViewBag.Technology)
    {
        <li>@tech</li>
    }
</ul>
```

Note:

If you're accessing a collection (like a list), you should cast it to the expected type for iteration to avoid runtime errors: Since **ViewBag** is a dynamic type, if you mistakenly assign a different data type, accessing it without typecasting could lead to runtime errors or unexpected behavior.



TempData

- **Description:** A dictionary for storing data temporarily, which survives one redirect.
- **Use:** Useful for passing messages between actions (e.g., success messages after form submissions).
- **Typecasting:** Required when retrieving data, as values are stored as **object**.

```
// In Controller
```

```
TempData["SuccessMessage"] = "Your data has been saved!";  
return RedirectToAction("Index");
```

```
// In View (after redirect)
```

```
<h1>@TempData["SuccessMessage"]</h1> // No typecasting needed if it's a string.
```

```
// If you store an integer:
```

```
TempData["Count"] = 5;
```

```
// In View (need to cast)
```

```
<h1>Count: @(Convert.ToInt32(TempData["Count"]))</h1> // Typecasting needed
```



Session Configuration in Program.cs

```
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30); // Set session timeout
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});
var app = builder.Build();

.....
app.UseSession();
.....
```



Session

- Session is an object derived from the HttpSessionState class.
- Session is the property of the HttpContext class.
- Session can also pass the data from the controller to the view and Action to Action.
- Session is valid for all requests, not for a single redirect and it is also need typecasting to getting data and check for null values to avoid error.
- Description: A way to store user-specific data for the duration of the session.
- Use: Suitable for storing user preferences, authentication states, or data across multiple requests.
- Typecasting: Required when retrieving data, as values are stored as object.



Session

```
public class HomeController : Controller
{
    public ActionResult Index()
    {
        HttpContext.Session.SetString("EmployeeName", "Asha Mehta");
        return View();
    }
}
```

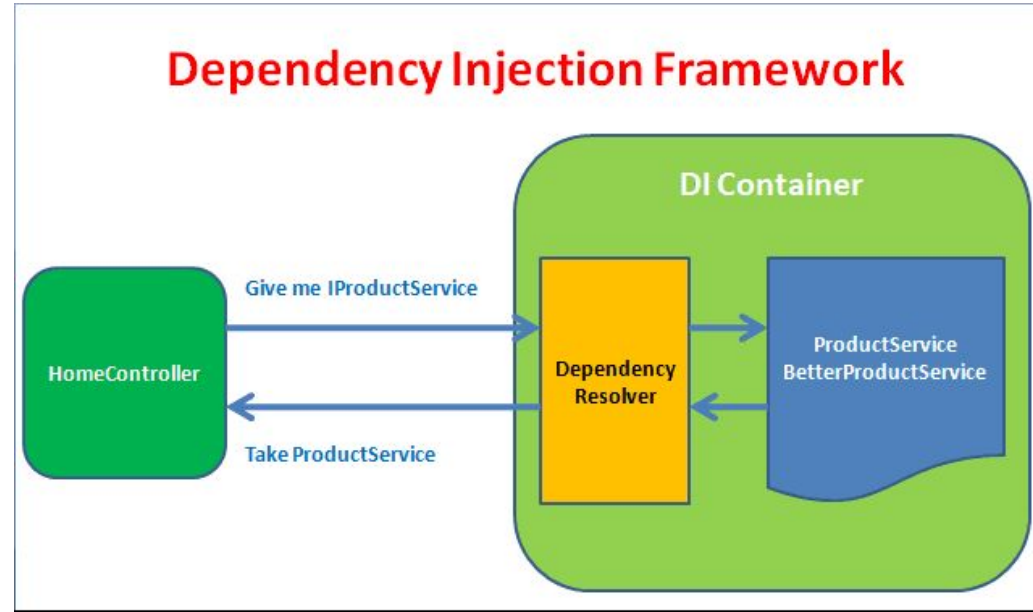
.....

```
Employee Name :- @Context.Session.GetString("EmployeeName");
```



Dependency Injection

- **Dependency Injection** (shortform "DI") is an ASP.NET Core technique to achieve loosely coupling between objects so that the applications can be maintained in an easy manner.
- **Loose coupling** is a good object-oriented programming practice of separation of concerns by allowing components to have little or no information of the internal workings and implementation of other components. Communication is done through interfaces. This approach allows for an easy substitution of components without many changes to the entire code base.



Dependency Injection

Types of Services in ASP.NET Core

Framework Services: Services that are a part of the ASP.NET Core framework such as `IApplicationBuilder`, `IHostingEnvironment`, `ILoggerFactory`, etc.

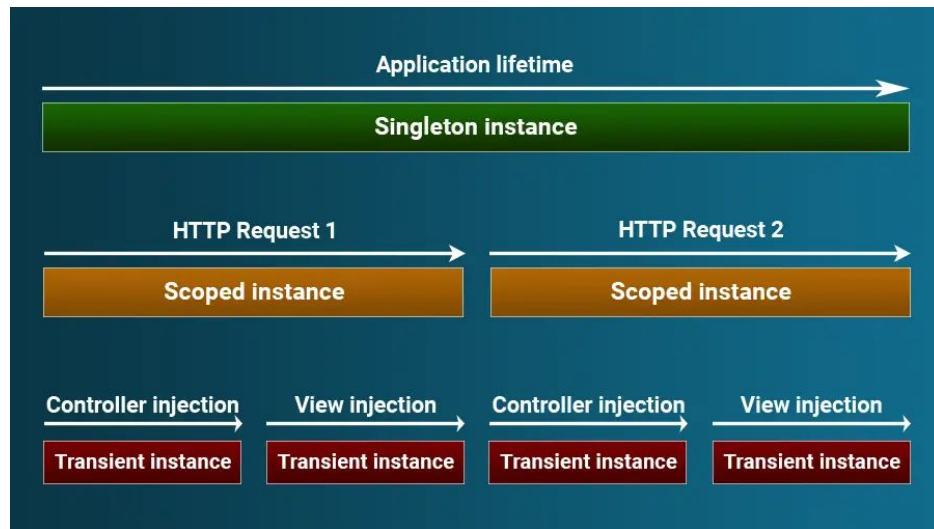
Application Services: The services (custom types or classes) which you as a programmer create for your application.

Different methods ASP.NET Core Provides to register a service with DI

Singleton: In this case, the IoC container will create and share a single instance of a service object throughout the application's lifetime.

Transient: In this case, the IoC container will create a new instance of the specified service type every time you ask for it.

Scoped: In this case, the IoC container will create an instance of the specified service type once per request and will be shared in a single request.



Dependency Injection

Example

Registering the Repository with ASP.NET Core Dependency Injection

```
//program.cs
```

```
builder.services.AddSingleton<IStudentRepository, TestStudentRepository>();
```

```
builder.services.AddTransient<IStudentRepository, TestStudentRepository>();
```

```
builder.services.AddScoped<IStudentRepository, TestStudentRepository>();
```

```
//Constructor Injection in Controller
```

```
public class HomeController : Controller
```

```
{
```

```
private readonly IStudentRepository _repository = null;
```

```
public HomeController(IStudentRepository repository)
```

```
{
```

```
    _repository = repository;
```

```
}
```

```
public JsonResult Index()
```

```
{
```

```
}
```



